Universiteit Antwerpen
Bachelor Informatica / Academic year 2026–2027

Emergency Response System

Artificial Intelligence Practicum

Assignment 1

A decision-support system for dispatching responders
under uncertain road and traffic conditions

Author	**Abdellah El Moussaoui**
Student number	20246031
Repository	https://github.com/abems123/ai-assignment-1
Date	12 November 2026

1 System design and probabilistic inference

Purpose, inputs and reproducibility

The system assigns emergency responders to incidents after a storm and selects routes through a directed city graph. I use the supplied distances and speeds, the responders' listed capabilities and availability, and incidents with release timestamps, priorities, arrival deadlines and handling durations. All times are seconds; distances are metres and speeds metres per second. An incident becomes eligible only when its timestamp is reached. Its deadline is an absolute latest arrival time; handling can finish later.

Running `python3 run.py X`, with `X` equal to 1, 2 or 3, produces `plan.json` and `statistics.txt` in `outputs/scenarioX/`. Python 3.9 or later is sufficient. The implementation uses only the standard library, including `csv`, `json`, `heapq`, `dataclasses`, `math` and `unittest`. The modules in `ers/` separate inference, routing, allocation, simulation and validation.

Bayesian reasoning: road accessibility

In `ers/bayesian.py`, each directed road has a hidden status: Open, Congested or Blocked. Citizen, police and sensor reports are noisy evidence. For a candidate status s, local weather w, and reports r_j, I calculate weights and normalize them:

$$\text{weight}(s) = P(s \mid w) \prod_j P(r_j \mid s, \text{source}_j), \qquad P(s \mid w, \mathbf{r}) = \frac{\text{weight}(s)}{\sum_u \text{weight}(u)}.$$

The source-specific CPT rows are true statuses and the columns reported statuses. Reports are assumed conditionally independent given the true status. When local weather is unknown, the prior is marginalized as $\sum_w P(w)P(s \mid w)$; weather from another road is not copied. Reports in opposite directions are kept separate. I use the maximum a posteriori (MAP) status; exact ties prefer Blocked, then Congested, then Open. Blocked roads are excluded from routing, while Congested roads receive the specified 1.5 road-delay factor.

Hidden Markov Model: historical traffic

The HMM in `ers/hmm.py` estimates Low, Medium and High traffic. These levels are distinct from road accessibility. `observed_traffic.csv` contains historical observations, whereas `road_reports.csv` supplies the Bayesian road evidence. I sort each road's traffic sequence by `observation_nr`; these numbers are sequence indices, not simulation timestamps.

Forward filtering starts with the supplied initial distribution and updates it using the first emission. Before each subsequent observation, it predicts using the transition matrix, then applies the emission likelihood and normalizes. For a traffic level ℓ, transition matrix T, emission matrix E and observation o:

$$b_{\text{pred}}(\ell) = \sum_{\ell'} b(\ell') T_{\ell',\ell}, \qquad b_{\text{new}}(\ell) = \frac{E_{\ell,o} b_{\text{pred}}(\ell)}{\sum_u E_{u,o} b_{\text{pred}}(u)}.$$

The matrices are supplied, not learned from observation frequencies. I treat the last historical posterior as the belief at simulation time zero and keep it fixed. Unobserved roads use that scenario's initial distribution. This explicitly avoids inventing a duration for an HMM step or assuming a stationary distribution.

2 Travel costs, planning and execution

My revision of the generated travel-time model

Let $T_0 = d/v$ be normal travel time and let α be 1 for Open or 1.5 for Congested. The expected traffic multiplier uses the full HMM belief:

$$\overline{m} = P(\text{Low}) \cdot 1.0 + P(\text{Medium}) \cdot 1.25 + P(\text{High}) \cdot 1.5.$$

The initial LLM-generated implementation multiplied both delays. I changed `estimated_travel_time_seconds` to take their maximum, while preserving the early infinite-cost return for Blocked roads:

$$T_{\text{old}} = \alpha T_0 \overline{m}, \qquad \boxed{T_{\text{new}} = \max(\alpha T_0,\, T_0 \overline{m})}.$$

This models the road and traffic estimates as potentially overlapping congestion effects. For example, a normal 20-second trip with a Congested status and certain High traffic changes from $20 \cdot 1.5 \cdot 1.5 = 45$ seconds to 30 seconds. The factors 1.0, 1.25 and 1.5 and the maximum rule are my accepted modelling choices, rather than additional rules prescribed by the assignment.

Search: uniform-cost routing

`ers/routing.py` builds an adjacency list of usable directed roads with these fixed travel-time costs. Uniform-cost search uses a minimum-priority queue, stores the best accumulated cost and predecessor for each location, and stops when the destination is removed at minimum cost. It therefore finds a fastest route under the selected non-negative estimated costs. It needs no heuristic. An unreachable destination yields no feasible assignment; a responder already at the incident has a one-node path and zero travel time.

Constraint-based allocation

In `ers/allocation.py`, variables are known, pending incidents. Their domains contain currently feasible responders and an unassigned option. Domain checks enforce the required capability, current availability, a reachable route and arrival by the deadline. A shared set of responder identifiers prevents assigning one responder twice in a batch. Plain backtracking tries choices, undoes them and compares complete combinations; I chose it over forward checking for simplicity.

The objective is lexicographic: maximize Critical assignments, then High, Medium and Low, and finally minimize total response time. Response time is arrival minus incident timestamp, including waiting before dispatch. This is a joint optimizer for the current batch. It does not reserve units for unrevealed incidents or optimize sequences of future jobs.

Event-driven simulation and output

`ers/simulation.py` advances directly to incident appearances and responder releases. All events at the same time are processed before a joint allocation; pending incidents are reconsidered at each event. Units remain occupied during travel and handling, without interruption. On completion they stay at the incident location. Initially Busy units become free at their supplied locations after their remaining busy time. I intentionally omit automatic hospital transport and return-to-base journeys from this initial model.

The simulation ends when no future events remain, including all job completions. Remaining incidents are unhandled. Each exported assignment contains exactly `responder_id`, `incident_id`, `path` and its start `time`. An independent plan replay checks the output before it is saved.

3 Evaluation, collaboration and critical reflection

Testing and plan validation

I evaluated all three supplied scenarios and ran `python3 -m unittest discover -s tests -v` (61 tests). Checks cover hand-calculated Bayesian and HMM posteriors, all 81 hidden histories for a four-observation HMM example, and UCS against an independent Floyd–Warshall reference for all 845 location pairs. Simulation tests cover simultaneous events, initial busy time, handling before reuse, zero-duration jobs and exclusion of future information.

The independent replay in `ers/validation.py` checks schema, identifiers, visibility, capabilities, initial availability, directed routes, arrival deadlines, duplicate incidents and responder overlap during travel and handling. Arrival and completion times are recomputed, and invalid-plan mutations are rejected. All three current scenario plans pass validation.

Results and the effect of my change

Here, "success rate" means on-time handled incidents divided by *all* incidents; unhandled incidents count as deadline misses. Average response time includes waiting and travel but averages handled incidents only. The unit count includes each responder only once.

Scenario	Handled	Units used	Mean response (s)	Deadlines met
1	6/7	4	89.765	85.714%
2	9/12	7	36.982	75.000%
3	12/20	7	628.279	60.000%

Holding the other choices fixed, the initial multiplicative version handled only 3 of 12 incidents in Scenario 2: **25%**. After my change to `max(road_time, traffic_time)`, it handled 9 of 12: **75%**, an increase of 50 percentage points. More trips became feasible before their deadlines. Its handled-only mean rose from 15.419 to 36.982 seconds; these averages cover different sets of incidents and must be read with coverage.

LLM collaboration and test findings

ChatGPT/Codex, **GPT-6.1 sol**, explained concepts, proposed alternatives and generated the initial modules, simulator, validation and tests. It helped debug code, interpret scenario data and draft this report. I approved the choices above for simplicity and reproducibility, then revised the cost rule myself. Page 5 reproduces the exact first prompt. `docs/development_notes.md` records interactions, approved assumptions and initial tests; the code and README document the revision. I remain responsible for understanding and defending the implementation.

A legacy traffic-cost test expected 45 seconds under the old multiplication rule. It has been renamed and updated to expect 30, with low, mixed and high traffic subcases. Existing tests cover the Open-road traffic delay and exclusion of Blocked roads. All **61 tests now pass**. This tests the adopted cost policy; it does not prove real-world accuracy.

Limitations and next improvements

The 75% result validates consistency under the chosen estimates, rather than real road accuracy. The maximum rule may underestimate independent delays; with these multipliers, traffic cannot further increase an already Congested road's cost. MAP discards residual blockage risk, correlated reports can overstate confidence, and traffic beliefs stay fixed. Batch-optimal CSP choices also need not produce a globally optimal schedule. I would next compare calibrated cost rules and allocation policies on additional cases, then introduce time-varying traffic only with a justified time-step model.

First substantive LLM prompt

```
Read `Assignment1.pdf` and the provided scenarios carefully, then help me build
    an initial working version of the Emergency Response System.

I want to make the important design decisions myself, especially regarding the AI
    algorithms, simulation behavior, assumptions, and planning strategies.
    Whenever a decision is not explicitly specified in the assignment, explain
    the available options briefly, give your recommendation, and let me choose
    before implementing it.

Keep the implementation simple, modular, and easy to understand. Follow the
    assignment requirements exactly, avoid unnecessary complexity, and work with
    me step by step rather than implementing the entire solution at once.

I wanna be able to run the program as follows: python3 run.py X with X being 1,2
    or 3 corresponding to the scenario number.
```